

CVIS

Connected Vehicle Intelligence System

Master Project Documentation

Course: Computer Communication & Network Security (CCNS)
Type: Academic Project — Industry-Inspired Prototype
Version: 2.3 (Phase 5 Ready)
Stack: ESP32 · FastAPI · SQLite · Ollama · Next.js
Protocols: HTTP REST + MQTT (runtime-switchable)
Security: HMAC-SHA256 · AES-256-GCM · API Key Auth

Architecture at a Glance

```
graph LR
    ESP32 -- "HTTP / MQTT" --> FastAPI
    FastAPI -- "Websocket" --> NextJS[Next.js UI]
    NextJS -- "Multi-factor Telemetry" --> OllamaAI[Ollama AI]
    ESP32 -- "HMAC-SHA256 | AES-256-GCM" --> OllamaAI
    subgraph NextJS_Endpoints [Next.js UI Endpoints]
        direction LR
        driver[/driver/]
        noc[/noc/]
        admin[/admin/]
    end
```

Contents

1	Project Overview	5
1.1	Project Identity	5
1.2	Core Objectives	5
1.3	Non-Goals	5
2	System Architecture	6
2.1	High-Level Architecture Diagram	6
2.2	Component Summary	7
3	Communication Layer	8
3.1	Protocol Overview	8
3.2	Packet Flow — HTTP	9
3.3	Packet Schema	9
4	Security Implementation	11
4.1	Security Layer Architecture	11
4.2	Device Registration Flow	11
4.3	HMAC-SHA256 Implementation	11
4.4	AES-256-GCM Encryption	12
4.5	Fault Codes	12
5	Vehicle Simulation Engine	13
5.1	Physics Model Overview	13
5.1.1	Load Factor Computation	13
5.1.2	Battery Drain Model	13
5.1.3	Thermal Model	13
5.1.4	Range Estimation	13
5.2	Vehicle Operating Modes	14
5.3	Physics Interdependency Map	14
6	AI Integration	15
6.1	Architecture	15
6.2	Two AI Surfaces	15
6.3	Prompt Engineering	15
6.4	AI Throttling	16
7	Business Model & Centralized AI Economics	17
7.1	The Core Thesis and Economic Rationale	17
7.2	The “Driver Intuition” Fallacy	17
7.3	Edge vs. Centralized AI: Comparative Cost-Benefit Analysis	18

7.4	Dual-Tier Commercial SaaS Framework	19
7.5	Return on Investment (ROI) for Fleet Operators	20
8	Database Schema	22
8.1	Schema Overview	22
8.2	Packets Table	22
8.3	Server Config Keys	23
9	Network Operations Center (NOC)	24
9.1	NOC Purpose	24
9.2	NOC Terminology Glossary	24
9.3	NOC Live Controls	25
9.4	NOC Packet Inspector Fields	25
10	Frontend Pages	26
10.1	Routing Architecture	26
10.2	Driver Dashboard (/driver)	26
10.3	NOC Dashboard (/noc)	27
10.4	Admin Console (/admin)	27
10.5	Mobile Views (/alphamobile, /betamobile, etc.)	27
11	Chaos & Simulation Middleware	28
11.1	Purpose	28
11.2	Middleware Pipeline	28
11.3	Tamper Detection Sequence	28
12	API Reference	29
12.1	Complete Endpoint Inventory	29
13	CCNS Syllabus Mapping	31
14	Startup & Deployment	32
14.1	Prerequisites	32
14.2	One-Click Startup	32
14.3	Port Map	32
15	Security Analysis	33
15.1	Threat Model	33
15.2	Cryptographic Strength	33
A	WebSocket Event Types	34
B	Fault Code Reference	35
C	Physics Engine — Key Constants	36

List of Tables

2.1	CVIS Component Inventory	7
3.1	Transport Protocol Comparison	8
3.2	Telemetry Payload Schema v1.0	10
4.1	Vehicle Fault Codes	12
5.1	Vehicle Operating Modes — Emergent Classification	14
6.1	AI Endpoints	15
7.1	Architectural and Economic Comparison: Edge AI vs. Centralized CVIS AI	19
7.2	CVIS Commercial Tier Architecture	20
8.1	Database Tables	22
8.2	Runtime Server Configuration Keys	23
9.1	NOC Terminology Reference	24
9.2	NOC Live Controls — Backend Effect Mapping	25
10.1	Frontend Routes	26
12.1	Full API Endpoint Reference	29
13.1	CVIS Features Mapped to CCNS Units	31
14.1	Software Prerequisites	32
14.2	Service Port Map	32
15.1	Threat Model and Mitigations	33
A.1	WebSocket Event Schema	34
B.1	Extended Fault Code Reference	35
C.1	Physics Engine Constants	36

List of Figures

2.1	CVIS High-Level System Architecture	6
-----	---	---

3.1	HTTP Telemetry Ingest Pipeline	9
4.1	CVIS Security Layer Stack	11
5.1	Physics Variable Interdependency Graph	14
11.1	ChaosMiddleware Execution Pipeline	28

Chapter 1

Project Overview

1.1 Project Identity

The **Connected Vehicle Intelligence System (CVIS)** is an academic prototype built for the *Computer Communication & Network Security (CCNS)* course. Its central thesis is that AI reasoning belongs in the cloud, not embedded in each vehicle. The vehicle is a *thin, securely-networked client*; the laptop running the backend acts as the data-centre. CVIS demonstrates the **secure communication layer** that makes this architecture viable.

Core Thesis

AI lives centrally (cloud/data-centre). The vehicle is a thin client. The project grades the **secure communication pipeline**, not the AI itself.

1.2 Core Objectives

1. Simulate a realistic EV telemetry pipeline from edge device to centralised server.
2. Enforce cryptographic integrity (HMAC-SHA256) and optional confidentiality (AES-256-GCM) on every packet.
3. Demonstrate runtime protocol switching between HTTP REST and MQTT publish-subscribe without restarting any service.
4. Provide a live Network Operations Center (NOC) dashboard that visualises packet flow, security events, and chaos simulations in real time.
5. Integrate Ollama AI reasoning over multi-factor telemetry to produce contextually accurate driver recommendations.

1.3 Non-Goals

- Real EV hardware or real sensors — telemetry is simulated on an ESP32 or via the Python physics engine.
- A generic chatbot with no telemetry grounding.
- Three separate frontend codebases (one unified Next.js app serves all three roles).

Chapter 2

System Architecture

2.1 High-Level Architecture Diagram

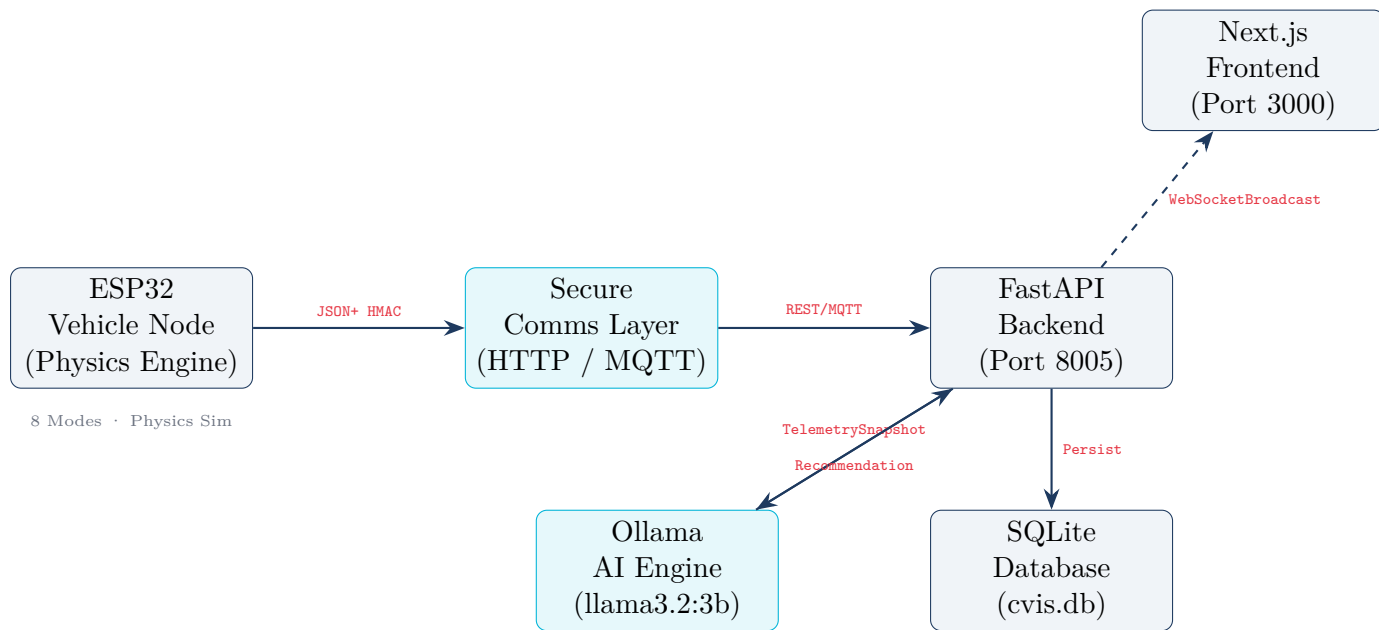


Figure 2.1: CVIS High-Level System Architecture

2.2 Component Summary

Table 2.1: CVIS Component Inventory

Component	Technology	Location	Role
ESP32 Firmware	Arduino C++	<code>/firmware</code>	Stateful physics engine; generates telemetry; signs with HMAC; supports HTTP + MQTT
Python Simulator	Python 3	<code>/backend/simulator.py</code>	Local vehicle physics emulator; same logic as firmware, used for development
FastAPI Backend	Python + FastAPI	<code>/backend</code>	Core server: auth, crypto, chaos middleware, AI bridge, WebSocket fan-out
SQLite Database	aiosqlite	<code>/backend/cvis_db</code>	Persists packets, telemetry, devices, auth logs, and server config
Ollama AI	llama3.2:3b / phi3-mini	Local Ollama	Multi-factor telemetry reasoning, driver recommendations, free-text chat
Next.js Frontend	Next.js + Tailwind	<code>/frontend</code>	Unified three-role UI: Driver, NOC, Admin

Chapter 3

Communication Layer

3.1 Protocol Overview

CVIS supports two fully-functional transport protocols, switchable at runtime with zero restarts:

Table 3.1: Transport Protocol Comparison

Property	HTTP REST	MQTT Pub/Sub
Pattern	Request-Response	Publish-Subscribe
Connection	Stateless, per-request	Persistent TCP via broker
Topic/Endpoint	POST /api/v1/telemetry	cvis/telemetry/{device_id}
Auth mechanism	X-API-Key header	_meta.api_key in envelope
HMAC transport	X-HMAC-Signature header	_meta.hmac in envelope
Config subscribe	Poll /api/v1/config/protocol	Subscribe cvis/config/#
Switch latency	Immediate (next packet)	MQTT broker must be running
Best used when	Demonstration, reliability	Bandwidth-constrained, IoT

3.2 Packet Flow — HTTP

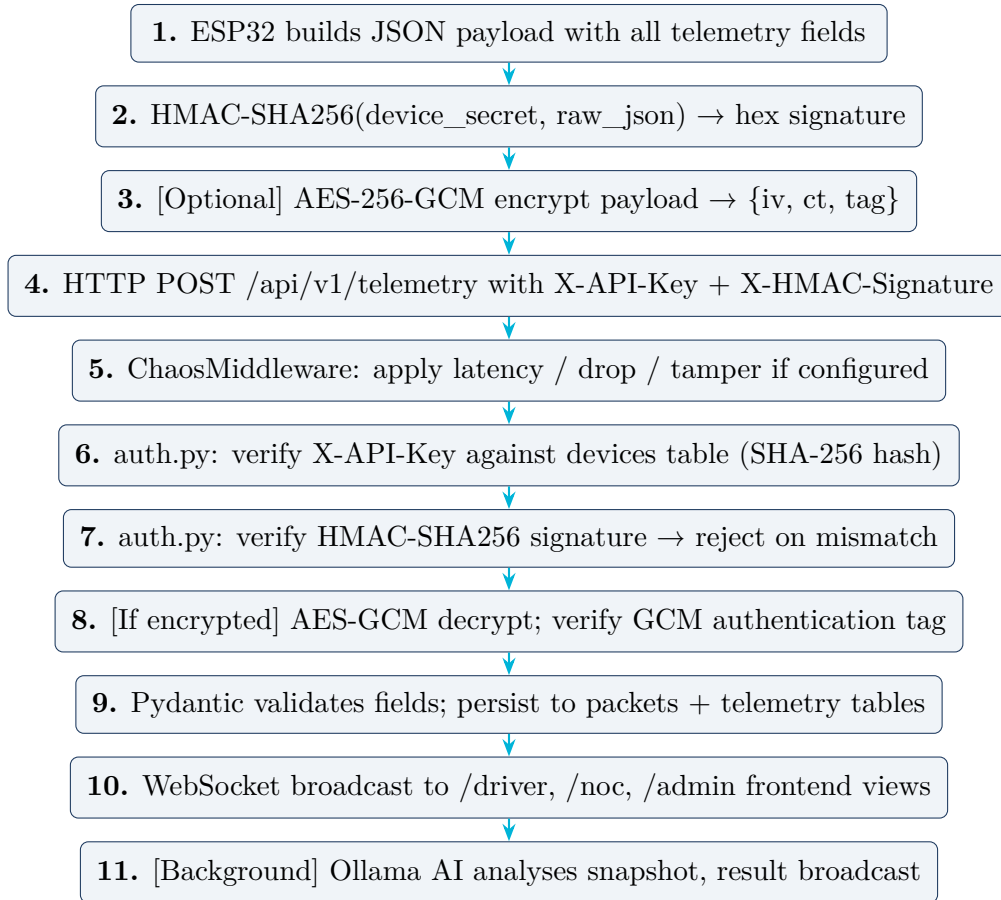


Figure 3.1: HTTP Telemetry Ingest Pipeline

3.3 Packet Schema

Every packet conforms to the following JSON schema (defined in `backend/models.py`):

Table 3.2: Telemetry Payload Schema v1.0

Field	Type	Required	Description & Bounds
device_id	string	Yes	Unique vehicle ID (1–64 chars)
schema_version	string	No	Schema version, default "1.0"
timestamp_ms	integer	Yes	ESP32 <code>millis()</code> at packet creation
mode	string	Yes	One of 8 allowed modes (see Table 5.1)
speed_kmh	float	No	Vehicle speed; $0 \leq v \leq 250$ km/h
battery_pct	float	No	State of charge; $0 \leq \text{SoC} \leq 100\%$
battery_temp_c	float	No	Battery pack temp; $-20 \leq T \leq 100^\circ\text{C}$
motor_temp_c	float	No	Motor temp; $-20 \leq T \leq 150^\circ\text{C}$
range_km	float	No	Estimated remaining range; $0 \leq r \leq 1000$ km
fault_code	integer	No	0x00 = none, 0x02 = overheat, 0x04 = motor
charging_rate_w	float	No	Charging power; 0 when not charging
<i>Advanced Physics Fields (Optional — Schema v2.3)</i>			
ambient_temp_c	float	No	Outside air temperature in $^\circ\text{C}$
headwind_kmh	float	No	Headwind speed in km/h
road_gradient_pct	float	No	Road gradient (+uphill / downhill)
tire_pressure_psi	float	No	Average tire pressure in PSI
cabin_climate_w	float	No	AC/Heater power draw in Watts
max_cell_voltage_diff_mv	float	No	Max voltage difference between cells (V)

Chapter 4

Security Implementation

4.1 Security Layer Architecture

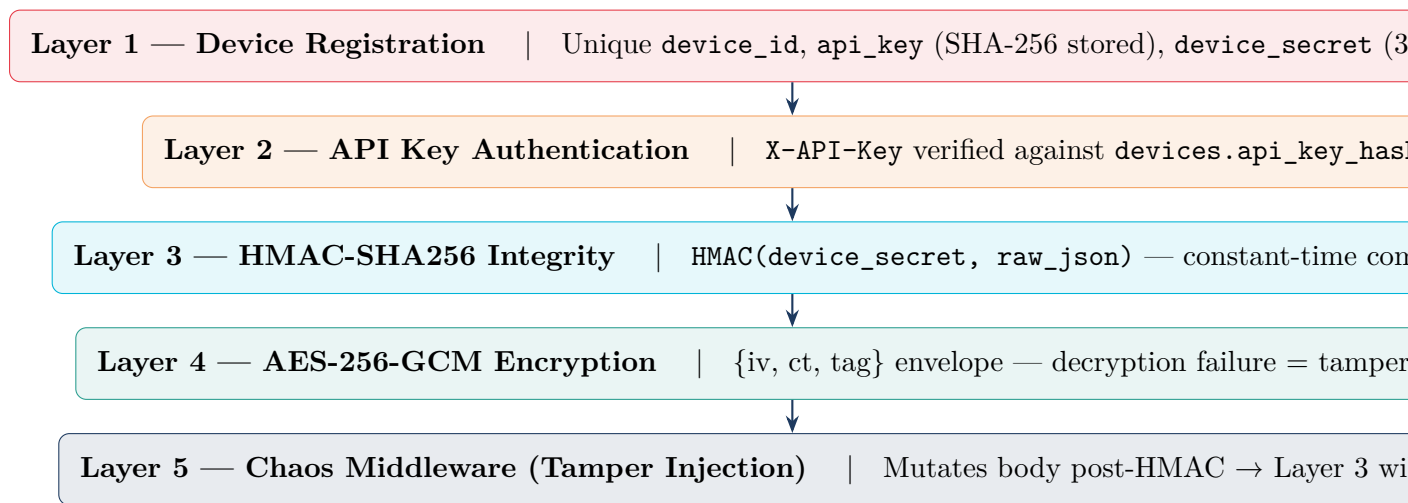


Figure 4.1: CVIS Security Layer Stack

4.2 Device Registration Flow

When the Python simulator or an ESP32 first connects:

1. POST /api/v1/devices/register with {"device_id": "SIM-001"}.
2. Backend generates: api_key = secrets.token_hex(32) and device_secret = secrets.token_hex(32).
3. api_key_hash = SHA256(api_key) is stored in devices table. The plaintext api_key is returned once.
4. All subsequent requests include X-API-Key: <plaintext> and X-HMAC-Signature: <hex>.

4.3 HMAC-SHA256 Implementation

HMAC is computed over the *exact raw JSON bytes* as transmitted. This is critical: even a single extra space in the JSON would change the HMAC and trigger rejection.

Listing 4.1: HMAC Computation (backend/crypto.py)

```

1 def compute_hmac(secret_hex: str, payload: str) -> str:
2     key = bytes.fromhex(secret_hex)          # 32-byte (256-bit) key
3     sig = hmac.new(key, payload.encode("utf-8"), hashlib.sha256)
4     return sig.hexdigest()                  # 64-char lowercase hex
5
6 def verify_hmac(secret_hex: str, payload: str, signature_hex: str) ->
    bool:
7     expected = compute_hmac(secret_hex, payload)
8     # hmac.compare_digest prevents timing-oracle attacks
9     return hmac.compare_digest(expected, signature_hex.strip().lower
        ())

```

4.4 AES-256-GCM Encryption

Listing 4.2: AES-GCM Encrypt/Decrypt (backend/crypto.py)

```

1 def aes_gcm_encrypt(secret_hex: str, plaintext: str) -> dict:
2     key = bytes.fromhex(secret_hex)          # 32 bytes = 256-bit AES key
3     aesgcm = AESGCM(key)
4     nonce = os.urandom(12)                   # 96-bit random nonce per
        message
5     ct_with_tag = aesgcm.encrypt(nonce, plaintext.encode("utf-8"),
        None)
6     ct = ct_with_tag[:-16]                   # ciphertext bytes
7     tag = ct_with_tag[-16:]                  # 128-bit GCM authentication
        tag
8     return {"iv": nonce.hex(), "ct": ct.hex(), "tag": tag.hex()}

```

The GCM **authentication tag** doubles as an integrity check: decrypting with a mismatched key or tampered ciphertext raises `InvalidTag`, which the backend logs as a `tamper_detected` event.

4.5 Fault Codes

Table 4.1: Vehicle Fault Codes

Code (hex)	Name	Description & Physics Trigger
0x00	No Fault	Normal operation
0x02	Battery Overheating	Battery temp > 60°C — caused by high discharge rate under load
0x04	Motor Fault	Motor temp > 95°C — caused by high speed + uphill gradient + drag

Chapter 5

Vehicle Simulation Engine

5.1 Physics Model Overview

The simulator is a **stateful physics engine**. Each tick (every 2 seconds) computes the next state based on the current state. No values are hardcoded per mode; modes are *emergent* from the physics.

5.1.1 Load Factor Computation

The **Load Factor** (L) is the normalised measure of how hard the vehicle is working at any instant:

$$L = \underbrace{\frac{v}{50}}_{\text{speed drag}} + \underbrace{\frac{(v+w)^2}{10000}}_{\text{aerodynamic drag}} + \underbrace{g \cdot 0.5}_{\text{gravity drag}} + \underbrace{\max\left(0, \frac{36 - p_t}{10}\right)}_{\text{tire drag}} \quad (5.1)$$

where v = speed (km/h), w = headwind (km/h), g = road gradient (%), p_t = tire pressure (PSI).

5.1.2 Battery Drain Model

$$\Delta \text{SoC} = \left(L \cdot 0.1 + \frac{P_{\text{cabin}}}{5000} \right) \cdot \Delta t \quad (5.2)$$

5.1.3 Thermal Model

Motor Temperature:

$$\Delta T_{\text{motor}} = (L \cdot 2.0 - (T_{\text{motor}} - T_{\text{ambient}}) \cdot 0.05) \cdot \Delta t \quad (5.3)$$

Battery Temperature:

$$\Delta T_{\text{battery}} = (\Delta \text{SoC} \cdot 5.0 - (T_{\text{battery}} - T_{\text{ambient}}) \cdot 0.02) \cdot \Delta t \quad (5.4)$$

5.1.4 Range Estimation

$$r = \text{SoC} \times 3.0 \times f_{\text{tire}} \times f_{\text{wind}} \times f_{\text{cabin}} \quad (5.5)$$

where penalty factors are: $f_{\text{tire}} = 0.9$ if $p_t < 32$ PSI; $f_{\text{wind}} = 0.85$ if $w > 20$ km/h; $f_{\text{cabin}} = 0.95$ if $P_{\text{cabin}} > 1000$ W.

5.2 Vehicle Operating Modes

Table 5.1: Vehicle Operating Modes — Emergent Classification

Mode	Classification Trigger	Typical Characteristics
Healthy	$30 \leq v \leq 80$ km/h	Normal cruise; moderate load
Eco	(button-selected / low speed)	Conservative speed, max range
Sport	$v > 80$ km/h	High motor temp, fast drain
Heavy Traffic	$0 < v < 30$ km/h	Stop-start, cabin AC dominant
Low Battery	$\text{SoC} \leq 15\%$	Speed limited; urgent AI alert
Battery Overheating	$T_{\text{batt}} > 60^\circ\text{C}$	Fault 0x02; limp mode (40 km/h cap)
Charging	$v = 0$ and $P_{\text{charge}} > 0$	Battery replenishing
Motor Fault	$T_{\text{motor}} > 95^\circ\text{C}$	Fault 0x04; forced stop

5.3 Physics Interdependency Map

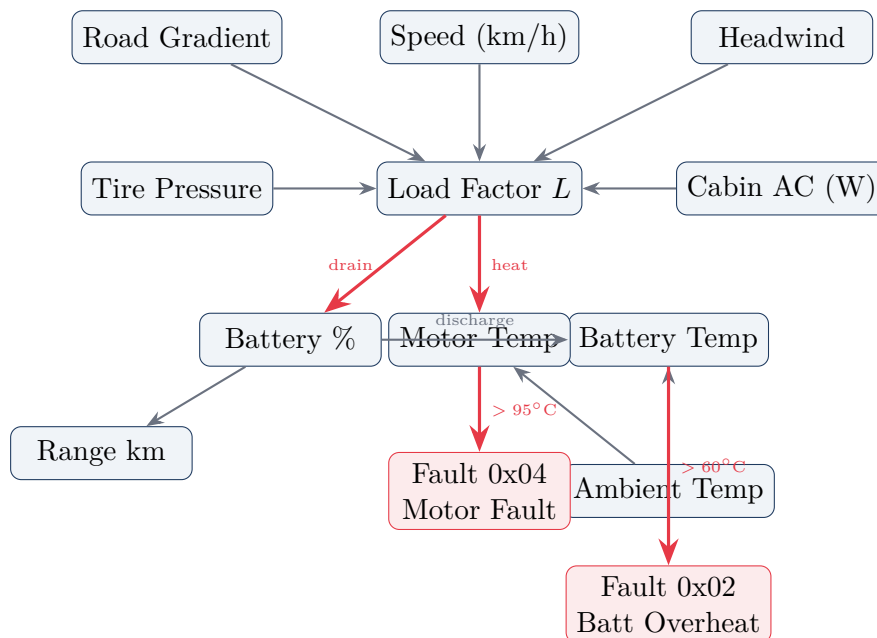


Figure 5.1: Physics Variable Interdependency Graph

Chapter 6

AI Integration

6.1 Architecture

CVIS uses **Ollama** running locally with `llama3.2:3b` (or `phi3-mini`). The AI is called via HTTP to `http://localhost:11434/api/chat`. The model is never called directly from the frontend — it is always mediated by the FastAPI backend, which ensures telemetry grounding.

6.2 Two AI Surfaces

Table 6.1: AI Endpoints

Surface	Endpoint	Behaviour
Auto Recommendation	Background task per telemetry update	Analyses combined snapshot; produces severity-tagged alert; broadcast via Web-Socket
Driver Chat	POST <code>/api/v1/ai/chat</code>	Free-text Q&A grounded in current + last 10 readings; cannot invent vehicle state

6.3 Prompt Engineering

The prompt is built in `backend/ai/prompt_builder.py`. The user message sent to Ollama contains:

1. **Current Snapshot:** all 17 telemetry fields from the latest packet.
2. **Advanced Physics Section:** ambient temp, headwind, road gradient, tire pressure, cabin load, cell voltage delta — labelled `=== ADVANCED PHYSICS ===`.
3. **Trend Block:** last 5 readings rendered as a sparkline text table (battery %, speed, motor temp, mode).
4. **Range Violations:** any field that has crossed its warning threshold.
5. **Mode Transition:** previous mode is explicitly shown when a mode change is detected.

The system prompt instructs the model to: (1) classify severity as **NORMAL** / **CAUTION** / **WARNING** / **CRITICAL**, (2) reason from the *combination* of factors rather than any single threshold, and (3) output exactly two lines: severity label + recommendation.

6.4 AI Throttling

To prevent LLM overload, the background AI task fires only when:

- At least 20 seconds have elapsed since the last inference, **OR**
- The vehicle mode has changed since the last packet.

Chapter 7

Business Model & Centralized AI Economics

7.1 The Core Thesis and Economic Rationale

The fundamental thesis of the Connected Vehicle Intelligence System (CVIS) states that **AI belongs centrally in the cloud/data-centre, not embedded per-vehicle**. The vehicle operates strictly as a *thin, securely-networked client*. This architectural decision is not merely a software convenience; it is a decisive commercial and engineering advantage.

Connected vehicles operate in an environment of rapid hardware commoditization, escalating computational demands, and strict automotive safety regulations. By shifting heavy model inference and state reasoning to centralized infrastructure, CVIS drastically lowers hardware Bills of Materials (BOM), removes vehicle thermal bottlenecks, eliminates vehicle software bricking hazards, and enables an economically viable Software-as-a-Service (SaaS) commercial business model.

7.2 The “Driver Intuition” Fallacy

A frequent industry objection to connected AI safety systems is the *Driver Intuition Fallacy*:

“Why should an operator pay for centralized AI when an experienced driver intuitively knows that high speeds and steep hills drain the battery and heat up the motor?”

While human intuition is capable of learning crude, qualitative heuristics (e.g., “driving fast drains the battery”), human cognition possesses severe and fatal limitations in modern electric vehicles:

1. **Reactive vs. Predictive Anomaly Detection:** Human drivers react exclusively to dashboard warning indicators, which illuminate only *after* a critical threshold is breached (e.g., Battery Temp > 60°C or Motor Temp > 95°C). At this point, irreversible cell degradation or forced vehicle immobilization has already begun. CVIS utilizes **Sliding-Window Trend Analysis** across consecutive telemetry packets to compute rate-of-change derivatives ($\Delta T/\Delta t$). The cloud AI detects thermal gradient anomalies rising 1.5°C/min faster than expected under current load, alerting the operator up to 15 minutes *before* any hardware threshold is tripped.
2. **Coupled Multi-Variate Physics vs. Gut Feeling:** EV range and thermal behavior are governed by non-linear coupled differential equations combining aerodynamics, road gradient, tire pressure, HVAC auxiliary draw, and cell impedance. If a driver asks:

“I have 24% battery remaining, ambient temperature is 38° C, headwind is 35 km/h, and I need to climb a 6% grade in stop-and-go traffic for 18 km with air conditioning active—will I reach the destination?”

Human intuition is mathematically incapable of answering this. CVIS acts as a real-time **Conversational Physics Simulator**, integrating all 17 live telemetry values against validated physical load models to provide a mathematically bounded, high-confidence guidance response.

3. **Fleet-Wide Swarm Intelligence:** An individual driver is isolated within their own vehicle envelope. Centralized AI aggregates telemetry across thousands of deployed nodes in real time. If 20 vehicles along a specific highway corridor experience anomalous battery heating or localized rolling-resistance spikes due to extreme ambient heat or road conditions, the central AI immediately flags predictive alerts to all upstream vehicles entering that corridor.

7.3 Edge vs. Centralized AI: Comparative Cost-Benefit Analysis

Embedding high-performance AI processors (e.g., specialized NPUs or high-power GPUs) inside every vehicle introduces severe technical and economic liabilities. Table 7.1 contrasts the embedded edge approach with the CVIS centralized architecture.

Table 7.1: Architectural and Economic Comparison: Edge AI vs. Centralized CVIS AI

Metric / Factor	Per-Vehicle Embedded AI	Centralized Cloud AI (CVIS)
Hardware BOM Cost	\$250–\$600 per vehicle (automotive NPU/SoC, heat sinks, shielding)	\$0 per vehicle (\$4 ESP32 thin client; shared cloud server amortized)
Thermal & Power Load	Draws 40W–150W continuous power; reduces total EV driving range by 5–10%	Zero vehicle power draw; all compute energy absorbed by cloud grid
Model Updating / OTA	High-risk multi-gigabyte OTA flashes; risk of bricking ECUs in the field	Instantaneous server-side model update; 0 vehicle downtime, 0 reflashes
Model Capacity	Severely constrained to quantized 1B–2B micro-models	Scalable 3B–70B parameter models (e.g., LLaMA 3.2, Phi-3, Nemotron)
Fleet Scaling Economics	Linear hardware cost scaling: $C_{\text{total}} = N \times C_{\text{hardware}}$	Sub-linear server cost scaling: shared inference queues and auto-scaling
Security & IP Surface	Attackers can physically probe CAN/SPI buses and dump proprietary model weights	Model weights protected in secure tier-4 data center behind HMAC auth
Cross-Fleet Correlation	Zero; nodes are completely siloed from one another	Full visibility; detects coordinated anomalies and systemic hardware faults
Latency	2–10 ms	50–250 ms (LAN / WAN round-trip)

As highlighted in Table 7.1, the single apparent advantage of embedded AI is raw inference latency (saving ≈ 150 ms). However, for supervisory safety guidance, range prediction, and operator advisory, a total round-trip time of 200–400 ms is completely imperceptible to human drivers and operational dispatchers. The massive capital savings, zero battery drain, and continuous model improvement make the centralized model overwhelmingly superior.

7.4 Dual-Tier Commercial SaaS Framework

CVIS realizes this economic advantage through a structured dual-tier subscription model:

Table 7.2: CVIS Commercial Tier Architecture

Tier	Target Market & Pricing	Included Features & SLA
Free Tier (<i>CVIS Community</i>)	Individual EV owners, academic researchers, small fleets (1–3 vehicles). Price: \$0 / month	• Plaintext HMAC-SHA256 integrity check • Standard Driver Dashboard (/driver) • 7-day rolling telemetry persistence • AI recommendations rate-limited (1 per 5 min) • Basic mobile app telemetry view • Max 2 registered devices
Premium Tier (<i>CVIS Fleet Pro</i>)	Commercial logistics fleets, taxi/ride-sharing networks, public transit, EV leasing operators. Price: \$15 / vehicle / month (\$150 / vehicle / year)	• Full AES-256-GCM payload encryption • Real-Time NOC Dashboard (/noc) with packet flow & live controls • Continuous real-time AI reasoning (<500 ms SLA) • Conversational Driver Chat with Neural Voice Synthesis (TTS) • Replay attack mitigation with timestamp validation • 12-month compliance telemetry retention • Chaos engineering network simulation suite • Cross-fleet anomaly detection & CSV audit export • Unlimited device registrations

7.5 Return on Investment (ROI) for Fleet Operators

For commercial fleet operators, subscribing to **CVIS Fleet Pro** yields immediate and quantifiable financial returns:

- **Battery Degradation Prevention:** EV battery replacements represent the single largest operational expense for fleet operators, costing \$7,000–\$15,000 per commercial pack. CVIS’s continuous multi-factor thermal and discharge monitoring prevents abusive thermal cycling, extending usable battery cycle life by 18%–25%. Over a 4-year lifecycle, this saves approximately \$2,500 per vehicle.
- **Downtime Reduction and Breakdown Avoidance:** Unscheduled roadside breakdowns cost logistics operators an average of \$450 in towing, recovery, and

missed delivery penalties. By detecting early mechanical and electrical faults (Fault 0x02 and 0x04) in advance, preventative maintenance is scheduled off-hours.

- **Insurance Premium Optimization:** Insurers offer 8%–14% discounts for fleets utilizing verifiable, cryptographically signed telemetry (HMAC-SHA256 audit trails) that prove adherence to speed, thermal, and maintenance safety standards.

Net Fleet Financial ROI

An annual investment of \$150/vehicle yields an estimated \$1,150/vehicle in avoided battery degradation, roadside recovery, and insurance optimization, representing an annual **7.6x Return on Investment (766% ROI)**.

Chapter 8

Database Schema

8.1 Schema Overview

All data is persisted in a single SQLite file (`backend/cvis.db`) using `aiosqlite` for non-blocking async access. The schema uses WAL (Write-Ahead Logging) for concurrent read-write.

Table 8.1: Database Tables

Table	Primary Key	Purpose
packets	id (auto)	Raw log of every inbound packet; protocol-agnostic
telemetry	id (auto)	Parsed, structured telemetry fields per packet
devices	device_id	Registered vehicle nodes with API key hash and secret
auth_logs	id (auto)	Every auth and tamper event
server_config	key	Key-value store for all runtime configuration
schema_version	version	Migration history tracker

8.2 Packets Table

Listing 8.1: Packets Table DDL

```
1 CREATE TABLE packets (  
2     id INTEGER PRIMARY KEY AUTOINCREMENT,  
3     received_at TEXT NOT NULL,  
4     device_id TEXT NOT NULL,  
5     protocol TEXT NOT NULL DEFAULT 'http', -- 'http' | '  
6         mqtt'  
7     direction TEXT NOT NULL DEFAULT 'inbound',  
8     size_bytes INTEGER,  
9     status TEXT NOT NULL, -- 'ok' | 'tamper_detected'  
10     ' | ...  
11     raw_json TEXT NOT NULL,  
12     encrypted INTEGER NOT NULL DEFAULT 0, -- 0 | 1  
13     encryption_method TEXT NOT NULL DEFAULT 'PLAIN', -- 'PLAIN' |  
14         'AES-GCM'  
15     auth_status TEXT NOT NULL DEFAULT 'ok'  
16 );
```

8.3 Server Config Keys

Table 8.2: Runtime Server Configuration Keys

Key	Default	Description
active_protocol	http	Current transport protocol (http/mqtt)
encryption_enabled	false	AES-256-GCM toggle
auth_enabled	true	API key + HMAC enforcement toggle
replay_protection	false	Timestamp-window replay protection
chaos_loss_pct	0	Packet loss rate (0/5/10/25%)
chaos_latency_ms	0	Artificial latency (0/100/300/1000 ms)
chaos_tamper	false	Payload mutation injector
mobile_app_enabled	true	Global mobile access gate
force_mode	""	Override vehicle simulation mode

Chapter 9

Network Operations Center (NOC)

9.1 NOC Purpose

The NOC at `/noc` is the centrepiece of the CVIS demonstration. It visualises every network event in real time and provides live controls that each trigger a *verifiable backend behaviour change*. No toggle is cosmetic.

9.2 NOC Terminology Glossary

Table 9.1: NOC Terminology Reference

Term	Definition
Packet	A single JSON payload from one vehicle tick, transported via HTTP POST or MQTT PUBLISH
RTT (Round-Trip Time)	Time from packet dispatch to acknowledgement receipt — artificially elevated by the Latency slider
HMAC	Hash-based Message Authentication Code — proves the payload was not modified in transit
AES-GCM	Advanced Encryption Standard in Galois/Counter Mode — provides both confidentiality and integrity
Auth Status	<code>auth_ok</code> <code>auth_fail</code> <code>no_auth</code> <code>tamper_detected</code>
Encryption Status	PLAIN (unencrypted) or AES-GCM (encrypted envelope)
Packet Loss %	Percentage of packets randomly dropped by the ChaosMiddleware before processing
Chaos Latency	Synthetic delay injected per packet in the middleware
Tamper Injection	Mutates a payload field <i>after</i> HMAC is computed, guaranteeing the backend rejects it
Protocol Switch	Live toggle between HTTP REST and MQTT; the backend broadcasts the change via MQTT so ESP32 nodes self-switch
Replay Attack	Re-sending an old captured packet to trick the backend — countered by timestamp-window protection
Limp Mode	Battery Overheating fault state: speed capped at 40 km/h by physics engine
Fan-out	One WebSocket broadcast from backend reaches <i>all</i> connected frontend tabs simultaneously

9.3 NOC Live Controls

Table 9.2: NOC Live Controls — Backend Effect Mapping

Control	API Endpoint	Real Backend Effect
Protocol Switch	POST <code>/api/v1/config/protocol</code>	Changes <code>active_protocol</code> in DB; broadcasts to MQTT <code>cvis/config/protocol</code> ; ESP32 self-switches on next poll
Encryption Toggle	POST <code>/api/v1/config/encryption</code>	Toggles AES-GCM; broadcasts to MQTT; ESP32 switches payload format on next packet
Auth Toggle	POST <code>/api/v1/config/auth</code>	Enables/disables API key + HMAC verification; traffic accepted without credentials in OFF mode
Packet Loss Slider	POST <code>/api/v1/config/chaos</code>	ChaosMiddleware probabilistically returns HTTP 503 before any processing
Latency Slider	POST <code>/api/v1/config/chaos</code>	<code>asyncio.sleep(latency_ms)</code> added before handler runs
Tamper Injector	POST <code>/api/v1/config/chaos</code>	Mutates <code>battery_pct=999.9</code> post-signature; next HMAC check will fail
Replay Protection	POST <code>/api/v1/config/replay</code>	Enables timestamp-window deduplication; duplicate packets rejected
Disconnect Vehicle	POST <code>/api/v1/control/disconnect</code>	Marks device inactive in DB; future packets from that device rejected
Stop AI Service	POST <code>/api/v1/control/ai</code>	Disables background AI task; recommendations paused globally

9.4 NOC Packet Inspector Fields

Each row in the NOC packet table can be expanded to reveal:

- **Packet ID** — sequential integer from `packets.id`
- **Source / Destination** — device ID → backend IP
- **Timestamp** — ISO 8601, backend receipt time
- **Protocol** — `http` or `mqtt`
- **Size (bytes)** — raw JSON body length
- **Auth Status** — `auth_ok` / `auth_fail` / `no_auth` / `tamper_detected`
- **Encryption** — `PLAIN` or `AES-GCM`
- **Fault Code** — hex fault from telemetry
- **AI Response** — last recommendation generated for this device
- **Raw JSON Payload** — full telemetry JSON (collapsible)

Chapter 10

Frontend Pages

10.1 Routing Architecture

The unified Next.js application (port 3000) provides:

Table 10.1: Frontend Routes

URL	Role	Primary Purpose
/	Landing	Role selector, project overview
/driver	Driver	Live vehicle telemetry, AI chat, health gauges, historical graphs
/noc	NOC Engineer	Live packet flow, control panel, security events
/admin	Administrator	System stats, auth logs, device registry, server health
/alphamobile	Premium Driver	Mobile view for ESP32-ALPHA (Premium tier)
/betamobile	Premium Driver	Mobile view for ESP32-BETA (Premium tier)
/gammamobile	Free Driver	Mobile view for ESP32-GAMMA (Free tier — gated)
/deltamobile	Free Driver	Mobile view for ESP32-DELTA (Free tier — gated)

10.2 Driver Dashboard (/driver)

Key Features

- Live telemetry gauges: speed, battery SoC, battery temp, motor temp, range, fault code
- Operating mode badge with colour coding
- AI Recommendation panel (severity-tagged: NORMAL / CAUTION / WARNING / CRITICAL)
- Historical trend graphs (Recharts library): battery%, motor temp, speed over time
- Driver Chat: free-text conversation grounded in live telemetry
- Active alerts panel driven by WebSocket events

Data flow: WebSocket `wss://api-cvis.justinsaju.me/ws` → event: `telemetry` message → React state update → re-render gauges.

10.3 NOC Dashboard (/noc)

Key Features

- Live animated packet flow diagram (ESP32 → Backend, colour-coded by protocol and security status)
- Packet table with 10 columns: ID, source, dest, timestamp, protocol, size, auth, encryption, fault, RTT
- Click-to-expand packet inspector (full JSON payload + AI response)
- All 9 live controls wired to real backend endpoints (Table 9.2)
- Live chaos metrics: total requests, dropped packets, tampered packets

10.4 Admin Console (/admin)

Key Features

- Connected vehicles list with status, tier, and last seen
- Server status: uptime, WebSocket client count, AI service health
- Packet statistics chart (last 12 hours by protocol)
- Auth log table: event type, device, IP, timestamp, details
- Failed request tracker and error log
- CPU/memory usage indicators

10.5 Mobile Views (/alphamobile, /betamobile, etc.)

Premium-tier vehicles (ESP32-ALPHA, ESP32-BETA) have a mobile-optimised view accessible at `/alphamobile` and `/betamobile`. Free-tier vehicles (GAMMA, DELTA) see a *gated* page. Access is controlled by the NOC's **Global Mobile App** toggle (`server_config.mobile_app_enabled`), which is verified server-side per device tier before granting access.

Chapter 11

Chaos & Simulation Middleware

11.1 Purpose

The `ChaosMiddleware` (in `backend/middleware/chaos.py`) simulates adverse network conditions at the **application middleware layer** rather than at the OS/network level. This is pedagogically equivalent and far simpler to set up in a demo environment.

11.2 Middleware Pipeline

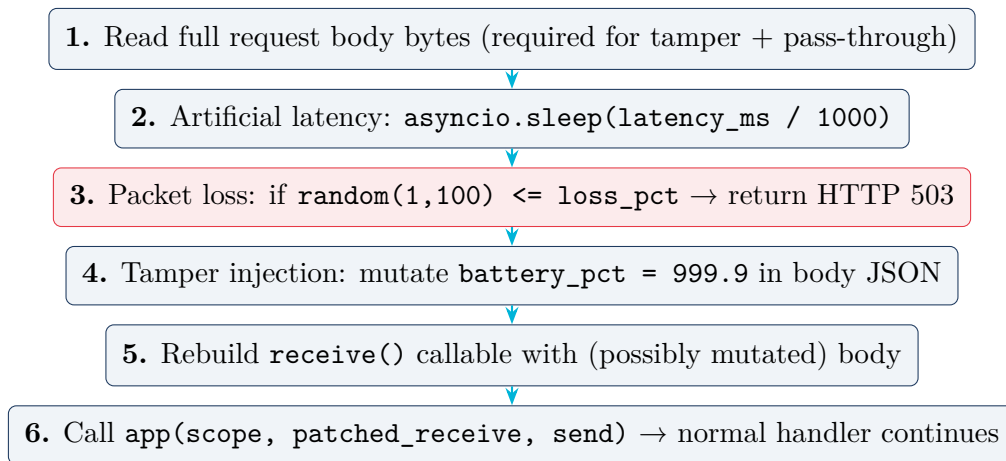


Figure 11.1: ChaosMiddleware Execution Pipeline

11.3 Tamper Detection Sequence

When **tamper injection** is enabled and **auth is enabled**:

1. ESP32 sends JSON with HMAC over the *original* payload.
2. ChaosMiddleware changes `battery_pct`: 999.9 in the body.
3. `auth.py` reads the (now mutated) body and recomputes HMAC.
4. HMAC mismatch detected → `tamper_detected` event logged to `auth_logs`.
5. Backend returns HTTP 401 with "HMAC signature verification failed".
6. NOC packet table shows `auth_status = tamper_detected` in red.

Chapter 12

API Reference

12.1 Complete Endpoint Inventory

Table 12.1: Full API Endpoint Reference

Endpoint	Method	Description
Telemetry		
/api/v1/telemetry	POST	Ingest telemetry packet (HTTP); requires X-API-Key + X-HMAC-Signature
/api/v1/telemetry/recent	GET	Fetch N most recent telemetry rows; optional device_id filter
/api/v1/telemetry/packets	GET	Fetch raw packet log for NOC table
Devices		
/api/v1/devices/register	POST	Register new device; returns api_key + device_secret
/api/v1/admin/deactivate-device	POST	Mark device as inactive
Configuration (NOC Controls)		
/api/v1/config/protocol	GET/POST	Get/set active protocol (http/mqtt)
/api/v1/config/encryption	GET/POST	Get/toggle AES-GCM encryption
/api/v1/config/auth	GET/POST	Get/toggle API key + HMAC enforcement
/api/v1/config/chaos	GET/POST	Get/update packet loss, latency, tamper settings
/api/v1/config/replay	GET/POST	Get/toggle timestamp replay protection
/api/v1/config/all	GET	Full config snapshot (all settings in one call)
/api/v1/config/mode	POST	Force a specific vehicle simulation mode
/api/v1/config/mobile-access	GET	Get mobile access state for a vehicle
/api/v1/config/mobile-app	POST	Globally enable/disable mobile app access
AI		
/api/v1/ai/recommendation	POST	On-demand AI recommendation for latest telemetry
/api/v1/ai/chat	POST	Driver chat grounded in current + historical telemetry
/api/v1/ai/status	GET	Check Ollama availability and model status
Admin		
/api/v1/admin/stats	GET	System stats: uptime, packet counts, auth counts
/api/v1/admin/devices	GET	Connected vehicle list with status and tier
/api/v1/admin/packet-stats	GET	Packet statistics over last N hours
/api/v1/admin/auth-logs	GET	Authentication and tamper event log
Control		
/api/v1/control/vehicles	GET	List all vehicles with real-time status

(Continued on next page)

(Continued from previous page)

Endpoint	Method	Description
/api/v1/control/disconnect	POST	Disconnect / deactivate a vehicle
/api/v1/control/ai	POST	Enable/disable AI service globally
WebSocket & System		
/ws	WS	WebSocket endpoint; fan-out broadcast to all connected clients
/health	GET	Backend health check
/docs	GET	Swagger UI auto-generated API docs

Chapter 13

CCNS Syllabus Mapping

Table 13.1: CVIS Features Mapped to CCNS Units

Unit	Topic	CVIS Feature	Where in Code
Unit 1	Network Topologies, Client-Server	ESP32 (thin client) → FastAPI (server); NOC, Admin, Driver roles	<code>main.py</code> , <code>ws_manager.py</code>
Unit 2	Application Layer Protocols, REST, MQTT Pub/Sub	HTTP REST + MQTT runtime-switchable; MQTT topics <code>cvis/telemetry/#</code> and <code>cvis/config/#</code>	<code>comms/mqtt_adapter.py</code> , <code>routers/config.py</code>
Unit 3	Reliability, RTT, Packet Loss, Latency, Retransmission	ChaosMiddleware: configurable loss, latency, tamper; ESP32 retry with exponential backoff	<code>middleware/chaos.py</code> , <code>firmware.ino</code>
Unit 4	Authentication, Integrity, Encryption	API Key Auth, HMAC-SHA256, AES-256-GCM, tamper detection logging	<code>auth.py</code> , <code>crypto.py</code> , <code>routers/devices.py</code>
Unit 4	Replay Attack Protection	Timestamp-window deduplication; same (<code>device_id</code> , <code>timestamp_ms</code>) rejected within window	<code>replay_protection.py</code> , <code>routers/config.py</code>
Unit 5	Cloud/Edge AI, Intelligent Networking	Ollama AI centralised on server; multi-factor reasoning; not embedded per-vehicle	<code>ai/ollama_client.py</code> , <code>ai/prompt_builder.py</code>
Unit 5	Real-time Data Streaming	WebSocket fan-out; single broadcast reaches all three frontend views simultaneously	<code>ws_manager.py</code> , <code>routers/ws.py</code>

Chapter 14

Startup & Deployment

14.1 Prerequisites

Table 14.1: Software Prerequisites

Software	Version	Purpose
Python	≥ 3.11	Backend and simulator runtime
Node.js	≥ 18	Next.js frontend
Ollama	Latest	Local AI model runtime
Mosquitto MQTT	≥ 2.0	Optional MQTT broker (needed for MQTT mode)
Arduino IDE / PlatformIO	Latest	For flashing ESP32 firmware

14.2 One-Click Startup

A platform-specific launcher script starts all services and opens browser tabs automatically:

- **macOS:** `start_cvis.command` (double-click in Finder)
- **Windows:** `start_cvis.bat` (double-click in Explorer)

Both scripts: (1) activate Python venv, (2) start FastAPI backend, (3) start Next.js frontend, (4) start Python physics simulator, (5) open four browser tabs: `/driver`, `/noc`, `/admin`, `/alphamobile`.

14.3 Port Map

Table 14.2: Service Port Map

Port	Service	Notes
8005	FastAPI Backend	REST + WebSocket
3000	Next.js Frontend	All three UI roles
11434	Ollama AI	HTTP API (<code>/api/chat</code>)
1883	Mosquitto MQTT	Optional

Chapter 15

Security Analysis

15.1 Threat Model

Table 15.1: Threat Model and Mitigations

Threat	CCNS Category	CVIS Mitigation
Packet tampering in transit	Integrity	HMAC-SHA256: any bit change fails verification; logged as <code>tamper_detected</code>
Payload eavesdropping	Confidentiality	AES-256-GCM encryption; ciphertext is opaque without 32-byte device secret
Impersonation / spoofing	Authentication	Per-device API key; SHA-256 stored (never plaintext)
Replay attack	Non-repudiation	Timestamp-window replay protection; duplicate (<code>device_id</code> , <code>ts</code>) rejected
Rogue device	Access Control	Device must register first; inactive devices are rejected
AI hallucination	Data Integrity	Prompt is strictly grounded in real telemetry; model cannot invent state

15.2 Cryptographic Strength

- **API Keys:** 256-bit random (`secrets.token_hex(32)`), stored as SHA-256 hash.
- **HMAC Keys** (`device_secret`): 256-bit random per device.
- **AES-GCM Nonces:** 96-bit random per message (the GCM standard); never reused.
- **Timing Attacks:** `hmac.compare_digest` used for constant-time comparison.

Appendix A

WebSocket Event Types

Table A.1: WebSocket Event Schema

Event Key	Payload Description
telemetry	Full telemetry snapshot for all 17 fields; broadcast on every successful ingest
ai_recommendation	{device_id, severity, recommendation} — broadcast after AI inference
packet_logged	{packet_id, device_id, protocol, auth_status, encrypted, size_bytes}
global_mobile_app_changed	{enabled: bool} — broadcast when NOC changes mobile access gate

Appendix B

Fault Code Reference

Table B.1: Extended Fault Code Reference

Code	Name	Physics Trigger	AI Response Expectation
0x00	No Fault	—	NORMAL severity unless other thresholds breached
0x02	Battery Overheating	$T_{\text{batt}} > 60^{\circ}\text{C}$	WARNING or CRITICAL; AI explains load factors causing heat
0x04	Motor Fault	$T_{\text{motor}} > 95^{\circ}\text{C}$	CRITICAL; AI recommends immediate stop; references speed + gradient

Appendix C

Physics Engine — Key Constants

Table C.1: Physics Engine Constants

Constant	Value	Use
Base drain coefficient	0.1	Battery% per unit load per second
Cabin drain coefficient	1/5000	Battery% per Watt per second
Motor heating coefficient	2.0	°C per unit load per second
Motor cooling coefficient	0.05	°C decay towards ambient per second
Battery heating coefficient	5.0	°C per unit drain per second
Battery cooling coefficient	0.02	°C decay towards ambient per second
Range base multiplier	3.0	km per 1% battery at ideal conditions
Motor fault threshold	95°C	Above → Fault 0x04 injected
Battery overheat threshold	60°C	Above → Fault 0x02 injected
Low battery threshold	15% SoC	Below → Mode = "Low Battery"
Limp mode speed cap	40 km/h	Applied when Fault 0x02 is active

Bibliography

- [1] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.
- [2] OASIS, “MQTT Version 5.0,” OASIS Standard, Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.pdf>
- [3] S. Baccari, M. Hadded, H. Ghazzai, H. Touati, and M. Elhadeif, “Anomaly Detection in Connected and Autonomous Vehicles: A Survey, Analysis, and Research Challenges,” *IEEE Access*, vol. 12, pp. 19250–19276, 2024, doi: 10.1109/ACCESS.2024.3361829.
- [4] S. U. A. Laghari, W. Li, S. Manickam, P. Nanda, A. K. Al-Ani, and S. Karuppayah, “Securing MQTT Ecosystem: Exploring Vulnerabilities, Mitigations, and Future Trajectories,” *IEEE Access*, vol. 12, pp. 139273–139289, Jun. 2024, doi: 10.1109/ACCESS.2024.3412030.
- [5] Y. Ma, X. Li, W. Shi, and Q. Cheng, “STCLA: An Efficient Certificateless Authenticated Key Agreement Scheme for the Internet of Vehicles,” *IEEE Transactions on Vehicular Technology*, vol. 73, no. 4, pp. 4830–4841, Apr. 2024, doi: 10.1109/TVT.2023.3334034.
- [6] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, Nat. Inst. Stand. Technol., Gaithersburg, MD, USA, Nov. 2007.
- [7] National Institute of Standards and Technology, “The Keyed-Hash Message Authentication Code (HMAC),” Federal Information Processing Standards Publication (FIPS PUB) 198-1, Jul. 2008.
- [8] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Dec. 2011.
- [9] S. Ramírez, “FastAPI: Modern, High-Performance Web Framework for Python,” 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- [10] Ollama Foundation, “Ollama: Local Large Language Model Architecture and Inference Engine,” 2024. [Online]. Available: <https://ollama.ai>
- [11] SQLite Consortium, “Write-Ahead Logging (WAL) in SQLite Database Engines,” 2023. [Online]. Available: <https://www.sqlite.org/wal.html>
- [12] Recharts Group, “Recharts: Redefined Chart Library Built with React and D3,” 2024. [Online]. Available: <https://recharts.org>
- [13] Espressif Systems, “ESP32 Technical Reference Manual: Cryptographic Hardware Acceleration,” Version 4.9, 2023.
- [14] A. Basiri et al., “Chaos Engineering,” *IEEE Software*, vol. 33, no. 3, pp. 35–41, May–Jun. 2016.